

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#2)

Write Lab Title Here

Student Name: Zuha Fatime

Roll No: 24I-6534

Section: CE-A

Date performed: 25,08, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____

Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad

Version: 1.0

Verified by: Dr. Nasir Ali Shah

Updated: Fall 2026

Task 1:

Description:

In this task, we made a Half Adder and Full Adder using Verilog. We used both Gate Level and Dataflow Level modeling. The Half Adder adds two binary inputs, while the Full Adder also uses a carry input. We wrote a testbench to check different input combinations. The results were observed in simulation and verified in logisim. The simulation showed that both adders were working correctly

Code:

// Paste source code here 

GATE LEVEL HALF ADDER

```
module half_adder(a,b,sum,carry);
input a,b;
output sum,carry;
xor (sum,a,b);
and (carry, a,b);
endmodule
```

module half_adder_tb;

```
reg a;
reg b;
wire sum;
wire carry;
```

```
half_adder uut(a,b,sum,carry);
```

```
initial
begin
a=0;b=0;
#50;
a=0;b=1;
#50;
a=1;b=0;
#50
a=1;b=1;
end
endmodule
```

DATA FLOW LEVEL HALF ADDER

```
module half_adder(a,b,sum,carry);
```

```
input a,b;
output sum,carry;
assign sum= a^b;
assign carry= a&b;
endmodule
module half_adder_tb;
reg a;
reg b;
wire sum;
wire carry;
half_adder uut(a,b,sum,carry);
initial
begin
a=0;b=0;
#50;
a=0;b=1;
#50;
a=1;b=0;
#50
a=1;b=1;
end
endmodule
GATE LEVEL FULL ADDER
module half_adder(a,b,sum,carry);
input a,b;
output sum,carry;
xor (sum,a,b);
and (carry, a,b);
endmodule
module full_adder(a,b,c,sum,carry);
input a,b,c;
output sum,carry;
wire s1,c1,c2;
half_adder ha1(a,b,s1,c1);
half_adder ha2(s1,c,sum,c2);
or(carry,c1,c2);
endmodule
module full_adder_tb;
reg a;
reg b;
```

```
reg c;
wire sum;
wire carry;

full_adder uut(a,b,c,sum,carry);
initial
begin
a=0;b=0;c=0;
#50;
a=0;b=0;b=1;
#50;
a=0;b=1;c=0;
#50
a=0;b=1;c=1;
#50
a=1;b=0;c=0;
#50;
a=1;b=0;b=1;
#50;
a=1;b=1;c=0;
#50
a=1;b=1;c=1;
end
endmodule
DATA FLOW LEVEL FULL ADDER
module half_adder(a,b,sum,carry);
input a,b;
output sum,carry;
assign sum=a^b;
assign carry=a&b;
endmodule
module full_adder(a,b,c,sum,carry);
input a,b,c;
output sum,carry;
wire s1,c1,c2;
half_adder ha1(a,b,s1,c1);
half_adder ha2(s1,c,sum,c2);
or(carry,c1,c2);
endmodule
module full_adder_tb;
reg a;
reg b;
reg c;
```

```

wire sum;
wire carry;

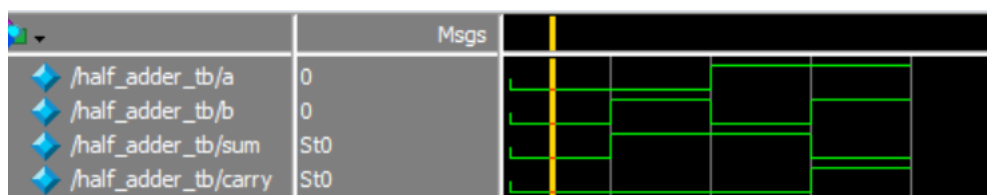
full_adder uut(a,b,c,sum,carry);
initial
begin
a=0;b=0;c=0;
#50;
a=0;b=0;b=1;
#50;
a=0;b=1;c=0;
#50
a=0;b=1;c=1;
#50
a=1;b=0;c=0;
#50;
a=1;b=0;b=1;
#50;
a=1;b=1;c=0;
#50
a=1;b=1;c=1;
end
endmodule

```

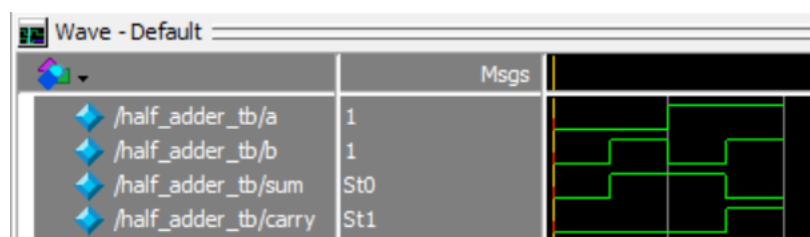
Output:

// Import output here 🖱️

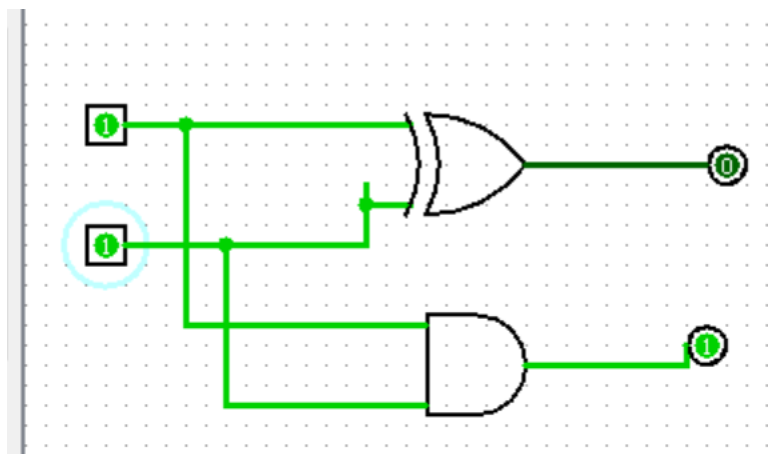
GATE LEVEL HALF ADDER



DATA FLOW LEVEL HALF ADDER

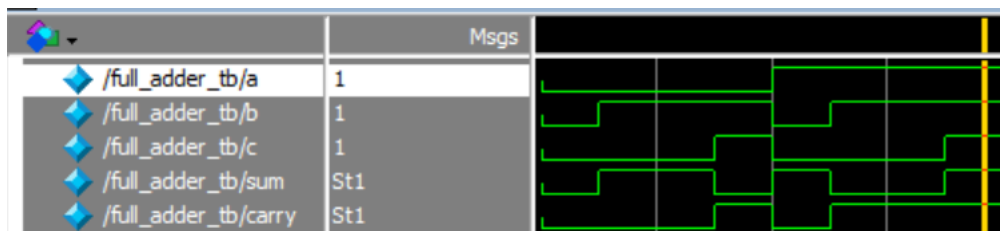


LOGISIM CIRCUIT

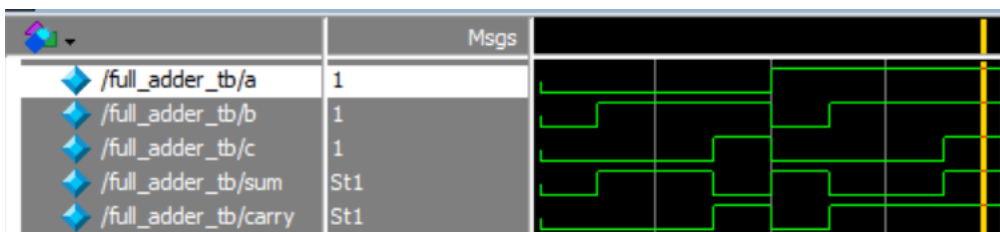


a	b	x	y
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

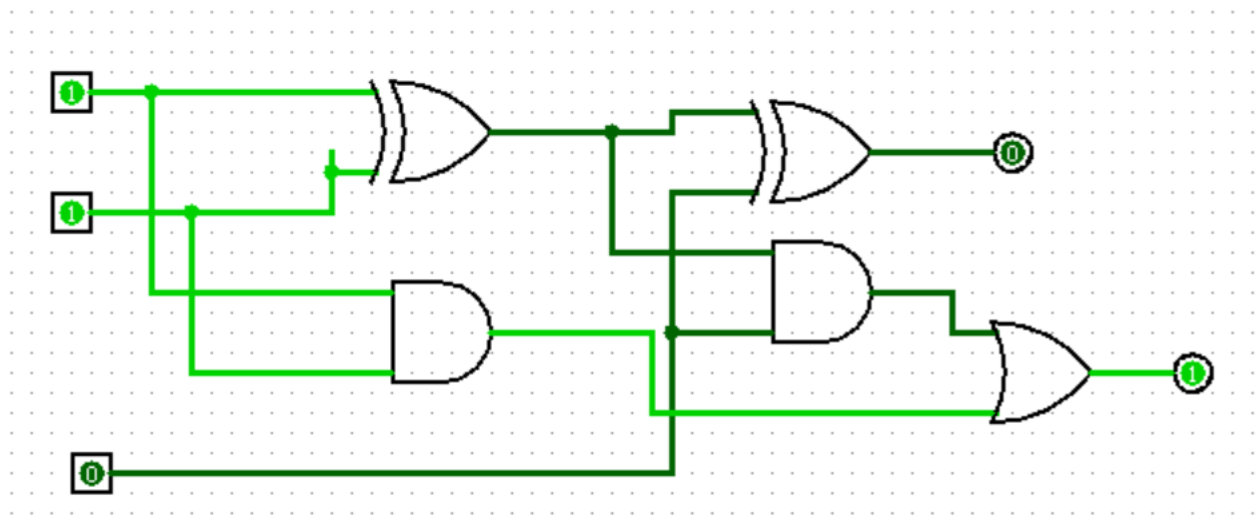
GATE LEVEL FULL ADDER



DATA FLOW LEVEL FULL ADDER



LOGISIM FULL ADDER



a	b	c	x	y
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Task 2:

Description:

// Write description of the task here 🖱 (max. five to seven lines)

In this task, we designed a 3×8 Decoder using Verilog. We used Gate Level and Dataflow Level modeling for the design. The decoder takes three inputs and gives eight outputs. For each input combination, only one output becomes active. We also wrote a testbench to check all the input combinations also verify in logisim. The simulation results showed that the decoder was working correctly

Code:

// Paste source code here 🖱

DECODER GATE LEVEL

```
module decoder3x8(a, b, c, en, y0, y1, y2, y3, y4, y5, y6, y7);  
input a, b, c, en;  
output y0, y1, y2, y3, y4, y5, y6, y7;  
wire na, nb, nc;  
not (na, a);  
not (nb, b);  
not (nc, c);  
and (y0, en, na, nb, nc);  
and (y1, en, na, nb, c);  
and (y2, en, na, b, nc);  
and (y3, en, na, b, c);  
and (y4, en, a, nb, nc);  
and (y5, en, a, nb, c);  
and (y6, en, a, b, nc);  
and (y7, en, a, b, c);  
endmodule
```

```
module decoder_tb;  
reg a;  
reg b;  
reg c;  
reg en;  
wire y0, y1, y2, y3;  
wire y4, y5, y6, y7;  
decoder3x8 uut(a, b, c, en, y0, y1, y2, y3, y4, y5, y6, y7);  
initial  
begin
```

```
en=0; a=0; b=0; c=0;
#50;
en=1; a=0; b=0; c=0;
#50;
a=0; b=0; c=1;
#50;
a=0; b=1; c=0;
#50;
a=0; b=1; c=1;
#50;
a=1; b=0; c=0;
#50;
a=1; b=0; c=1;
#50;
a=1; b=1; c=0;
#50;
a=1; b=1; c=1;
#50;
en=0;
#50;
end
endmodule

DECODE DATA FLOW LEVEL
module decoder3x8(a, b, c, en, y0, y1, y2, y3, y4, y5, y6, y7);
input a, b, c, en;
output y0, y1, y2, y3, y4, y5, y6, y7;
wire na, nb, nc;
assign na=~a;
assign nb=~b;
assign nc=~c;
assign y0=en&na&nb&nc;
assign y1=en&na&nb&c;
assign y2=en&na&b&nc;
assign y3=en&na&b&c;
assign y4=en&a&nb&nc;
assign y5=en&a&nb&c;
assign y6=en&a&b&nc;
assign y7=en&a&b&c;
endmodule

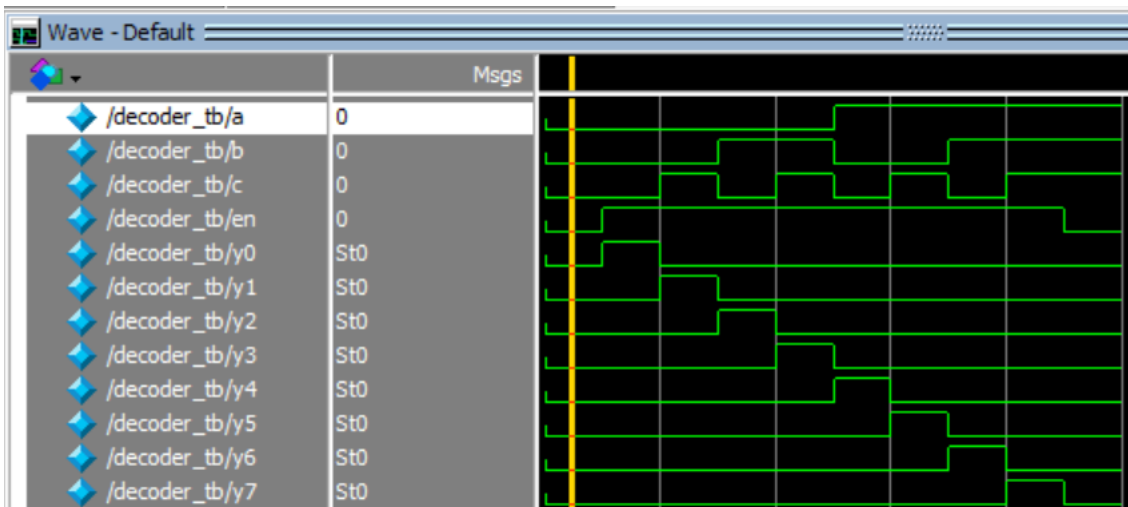
module decoder_tb;
reg a;
reg b;
reg c;
```

```
reg en;
wire y0, y1, y2, y3;
wire y4, y5, y6, y7;
decoder3x8 uut(a, b, c, en, y0, y1, y2, y3, y4, y5, y6, y7);
initial
begin
en=0; a=0; b=0; c=0;
#50;
en=1; a=0; b=0; c=0;
#50;
a=0; b=0; c=1;
#50;
a=0; b=1; c=0;
#50;
a=0; b=1; c=1;
#50;
a=1; b=0; c=0;
#50;
a=1; b=0; c=1;
#50;
a=1; b=1; c=0;
#50;
a=1; b=1; c=1;
#50;
en=0;
#50;
end
endmodule
```

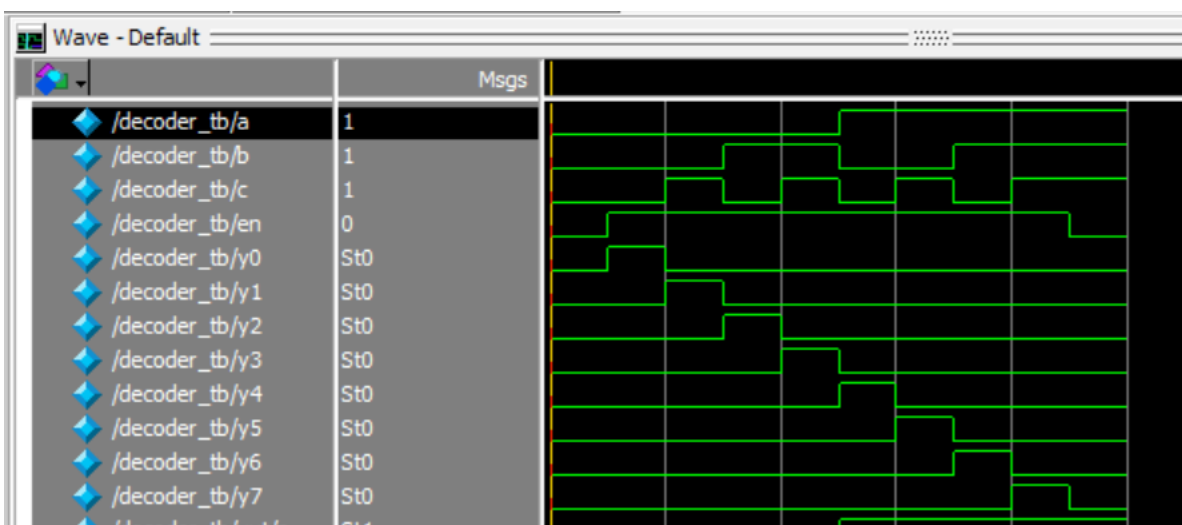
Output:

// Import output here 

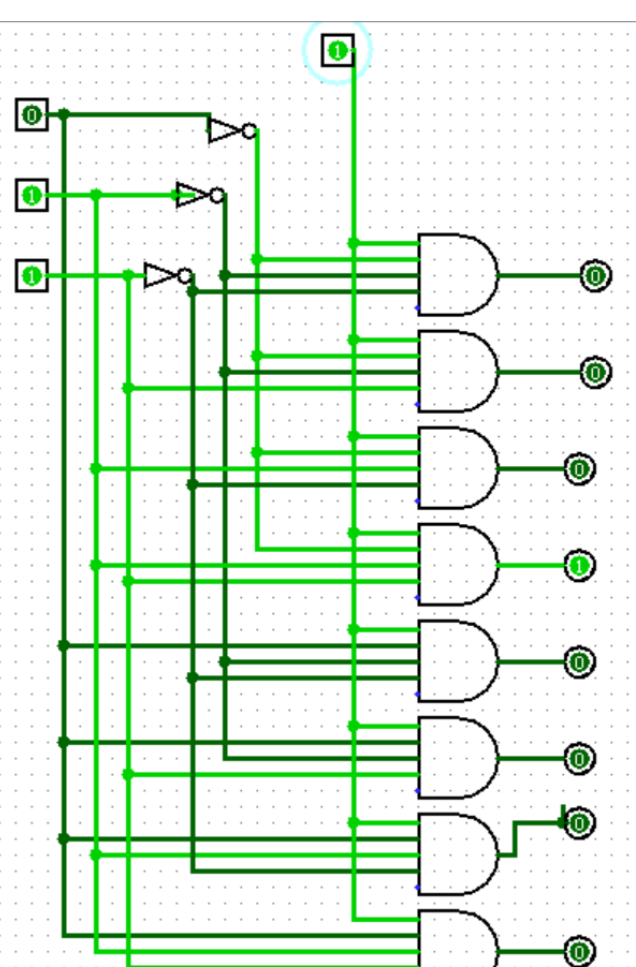
DECODER GATE LEVEL



DECODER DATA FLOW LEVEL



LOGISIM CIRCUIT



0	0	1	0	0	0	0	0	0	0	0	0
0	0	1	1	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	0	0
0	1	0	1	0	0	0	0	0	0	0	0
0	1	1	0	0	0	0	0	0	0	0	0
0	1	1	1	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	0	0	0	0
1	0	0	1	0	0	1	0	0	0	0	0
1	0	1	0	0	0	0	1	0	0	0	0
1	1	0	0	0	0	0	0	1	0	0	0
1	1	0	1	0	0	0	0	0	1	0	0
1	1	1	0	0	0	0	0	0	0	1	0
1	1	1	1	0	0	0	0	0	0	0	1

Build Circuit

Task 3:

Description:

// Write description of the task here 📝 (max. five to seven lines)

In this task, we designed an 8×1 Multiplexer using Verilog. We used both Gate Level and Dataflow Level modeling. The multiplexer has eight inputs and one output, and three select lines are used to choose the input. A testbench was written to test all the select combinations. We checked the output in the simulation also verify them using logisim. The results showed that the multiplexer was working correctly

Code:

GATE LEVEL

```
module mux_8x1( D0, D1, D2, D3, D4, D5, D6, D7, S2, S1, S0,Y);
input D0,D1,D2,D3,D4,D5,D6,D7,S2,S1,S0;
output Y;
wire ns2,ns1,ns0;
wire d0,d1,d2,d3,d4,d5,d6,d7;
not(ns2,S2);
not(ns1,S1);
not(ns0,S0);
and(d0,ns2,ns1,ns0,D0);
and(d1,ns2,ns1,S0,D1);
and(d2,ns2,S1,ns0,D2);
and(d3,ns2,S1,S0,D3);
and(d4,S2,ns1,ns0,D4);
and(d5,S2,ns1,S0,D5);
and(d6,S2,S1,ns0,D6);
and(d7,S2,S1,S0,D7);
or(Y,d0,d1,d2,d3,d4,d5,d6,d7);

endmodule

module mux_8x1_tb;
reg d0,d1,d2,d3,d4,d5,d6,d7;
reg s0,s1,s2;
wire Y;
mux_8x1 uut(d0,d1,d2,d3,d4,d5,d6,d7,s2,s1,s0,Y);
initial
begin
d0=0; d1=1; d2=0; d3=1;
d4=0; d5=1; d6=0; d7=1;
s2=0; s1=0; s0=0;
#50;
s2=0; s1=0; s0=1;
#50;
s2=0; s1=1; s0=0;
#50;
s2=0; s1=1; s0=1;
#50;
end
```

```
s2=1; s1=0; s0=0;
#50;
s2=1; s1=0; s0=1;
#50;
s2=1; s1=1; s0=0;
#50;
s2=1; s1=1; s0=1;
#50;

end

endmodule
DATA FLOW LEVEL
module mux_8x1( D0, D1, D2, D3, D4, D5, D6, D7, S2, S1, S0,Y);
input D0,D1,D2,D3,D4,D5,D6,D7,S2,S1,S0;
output Y;
wire ns2,ns1,ns0;
wire d0,d1,d2,d3,d4,d5,d6,d7;
assign ns2=~S2;
assign ns1=~S1;
assign ns0=~S0;
assign d0=ns2&ns1&ns0&D0;
assign d1=ns2&ns1&S0&D1;
assign d2=ns2&S1&ns0&D2;
assign d3=ns2&S1&S0&D3;
assign d4=S2&ns1&ns0&D4;
assign d5=S2&ns1&S0&D5;
assign d6=S2&S1&ns0&D6;
assign d7=S2&S1&S0&D7;
assign Y=d0|d1|d2|d3|d4|d5|d6|d7;

endmodule
module mux_8x1_tb;
reg d0,d1,d2,d3,d4,d5,d6,d7;
reg s0,s1,s2;
wire Y;
mux_8x1 uut(d0,d1,d2,d3,d4,d5,d6,d7,s2,s1,s0,Y);
initial
begin
d0=0; d1=1; d2=0; d3=1;
d4=0; d5=1; d6=0; d7=1;
s2=0; s1=0; s0=0;
#50;
s2=0; s1=0; s0=1;
#50;
s2=0; s1=1; s0=0;
#50;
s2=0; s1=1; s0=1;
```

```
#50;
s2=1; s1=0; s0=0;
#50;
s2=1; s1=0; s0=1;
#50;
s2=1; s1=1; s0=0;
#50;
s2=1; s1=1; s0=1;
#50;

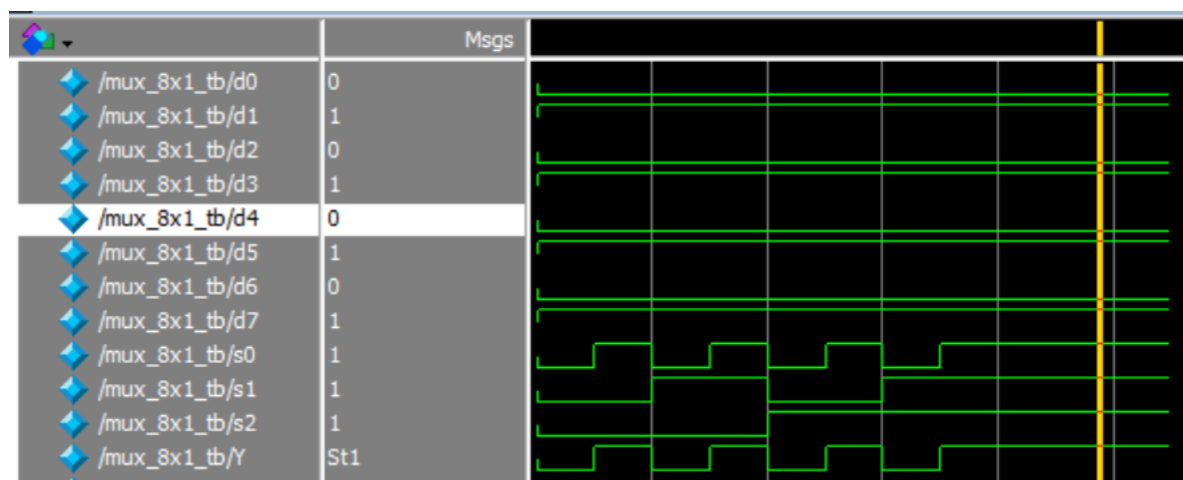
end

endmodule
```

Output:

// Import output here 🖱️

GATE LEVEL



DATA FLOW LEVEL

LOGISIM CIRCUIT

